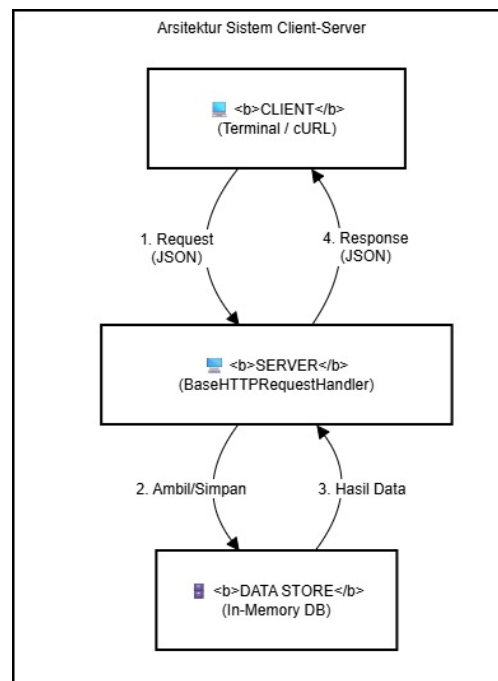




NAMA	KHOLID MASHURI
PRODI	D3-TEKNIK INFORMATIKA
KELAS	TI4A
MATKUL	UTS Sistem Terdistribusi & Keamanan Sistem Komputer

1. Buat diagram sederhana Client → Server → Data

Sistem ini menggunakan arsitektur dasar Client-Server berbasis REST API. Aliran data berjalan secara terdistribusi di mana komponen Client mengirimkan permintaan (*request*) melalui protokol HTTP, disusul oleh Server yang mengolah hak akses ke dalam penyimpanan data internal (*Data Store*) sebelum mengembalikan hasil (*response*).



2. Buat minimal 2 endpoint: POST /login dan GET /data

Sistem terdistribusi ini mengimplementasikan minimal 2 fungsi rute atau endpoint utama:

- **Endpoint 1: POST /login**
 - **Fungsi:** Digunakan untuk proses autentikasi (*authentication*). Menerima input data rahasia berupa username dan password dari client.
- **Endpoint 2: GET /data**
 - **Fungsi:** Digunakan untuk proses otorisasi (*authorization*). Endpoint ini mengunci data akademis mahasiswa dan hanya membukanya jika client membawa token yang sah.

Bagian Kode Endpoint /login

```
def do_POST(self):
    if self.path == "/login":
        panjang_data = int(self.headers.get("Content-Length", 0))
        isi_body = self.rfile.read(panjang_data).decode("utf-8")

        try:
            data_input = json.loads(isi_body)
            user_input = data_input.get("username")
            pass_input = data_input.get("password")

            # Memeriksa kecocokan data login hasil ketikan manual client
            if user_input == NAMA_USER_SAH and pass_input == PASSWORD_SAH:
                respon_sukses = {
                    "status": "success",
                    "token": TOKEN_RAHASIA
                }
                self._send_json(200, respon_sukses)
            else:
                respon_gagal = {
                    "status": "failed",
                    "message": "Username atau password salah"
                }
```

Bagian Kode Endpoint /data

```
def do_GET(self):
    if self.path == "/data":
        header_otorisasi = self.headers.get("Authorization")

        if header_otorisasi and header_otorisasi.startswith("Bearer "):
            try:
                token_kiriman = header_otorisasi.split(" ")[1]
            except IndexError:
                token_kiriman = ""

            # Memvalidasi token yang dimasukkan secara manual oleh client
            if token_kiriman == TOKEN_RAHASIA:
                respon_data = {
                    "status": "success",
                    "data": DATA_MAHASISWA
                }
                self._send_json(200, respon_data)
                return

            token_salah = {
                "status": "failed",
```

```

        "message": "Token tidak valid"
    }
    self._send_json(401, token_salah)
else:
    rute_salah = {"status": "failed", "message": "Endpoint tidak ditemukan"}
    self._send_json(404, rute_salah)

```

3. Simulasi JSON Request dan Response

A. Bagian Endpoint /login

- a. Simulasi JSON Request Login & Simulasi JSON Response Login Berhasil

```

mmp/htdocs/server py/client.py"
=== TAHAP 1: REQUEST LOGIN (AUTHENTICATION) ===
Masukkan Username: KHOLID GANTENG
Masukkan Password: KH0123

Response Login Berhasil (200 OK):
{
    "status": "success",
    "token": "token-kholid-12"
}

```

Penjelasan: Server sukses mencocokkan kredensial di dalam database internal, kemudian menghasilkan response sukses dengan menyertakan string pengenalan unik bernama token.

- c. Simulasi JSON Response Login Gagal (401 Unauthorized)

```

        "Jurusan": "Teknik Informatika"
    }
}
PS C:\xampp\htdocs\server py> & C:\Users\HP\AppData\Local\Microsoft\WindowsApps\python3.11.exe "c:/xa
mmp/htdocs/server py/client.py"
=== TAHAP 1: REQUEST LOGIN (AUTHENTICATION) ===
Masukkan Username: kholid gamteng
Masukkan Password: kho123

Login Gagal! Username atau Password yang Anda ketik salah.
PS C:\xampp\htdocs\server py>

```

Penjelasan: Kredensial yang diinputkan oleh client tidak cocok dengan data terdaftar sehingga server menolak pembuatan token akses.

B. Bagian Endpoint /data

- a. Simulasi JSON Request Data (*Dikirim melalui HTTP Authorization Header*)

HTTP

```

GET /data HTTP/1.1
Authorization: Bearer token-kholid-12

```

Penjelasan: Client meminta hak akses untuk membaca informasi rahasia dengan menyertakan token token-kholid-12 pada header HTTP sebagai bukti otorisasi.

- **b. Simulasi JSON Response Data Berhasil (200 OK)**

JSON

```
Masukkan atau tempel (paste) token Anda di sini: token-kholid-12

=== TAHAP 2: REQUEST DATA AKADEMIS (AUTHORIZATION) ===
Response Ambil Data Berhasil (200 OK):
{
  "status": "success",
  "data": {
    "Nama": "Kholid Mashuri",
    "NIM": "32241012",
    "Kelas": "TI 3A",
    "Jurusan": "Teknik Informatika"
  }
}
PS C:\xampp\htdocs\server py> █
```

Penjelasan: Validasi token pada sisi server berhasil lolos, sehingga server memberikan response sukses berisi objek data akademis lengkap milik Kholid Mashuri.

- **c. Simulasi JSON Response Data Gagal (401 Unauthorized)**

JSON

```
=== TAHAP 2: REQUEST DATA AKADEMIS (AUTHORIZATION) ===
Gagal Mengambil Data! Token manual yang Anda masukkan salah atau tidak valid.
PS C:\xampp\htdocs\server py> █
```

: Server mendeteksi bahwa token yang dikirimkan salah, kedaluwarsa, atau kosong, sehingga hak akses ke data mahasiswa ditolak.

4. Coding Python Sederhana Tanpa Framework

SERVER (server.py) :

```
from http.server import BaseHTTPRequestHandler, HTTPServer
import json

# Menyimpan konfigurasi database dalam variabel resmi agar editor tidak error
NAMA_USER_SAH = "KHOLID GANTENG"
PASSWORD_SAH = "KH0123"
TOKEN_RAHASIA = "token-kholid-12"

# Data akademis mahasiswa yang disimpan di server
DATA_MAHASISWA = {
    "Nama": "Kholid Mashuri",
    "NIM": "32241012",
    "Kelas": "TI 3A",
```

```

        "Jurusan": "Teknik Informatika"
    }

class RestAPIHandler(BaseHTTPRequestHandler):

    def _send_json(self, status_code, payload):
        self.send_response(status_code)
        self.send_header("Content-Type", "application/json")
        self.end_headers()
        self.wfile.write(json.dumps(payload, indent=4).encode("utf-8"))

    def do_POST(self):
        if self.path == "/login":
            panjang_data = int(self.headers.get("Content-Length", 0))
            isi_body = self.rfile.read(panjang_data).decode("utf-8")

            try:
                data_input = json.loads(isi_body)
                user_input = data_input.get("username")
                pass_input = data_input.get("password")

                # Memeriksa kecocokan data login hasil ketikan manual client
                if user_input == NAMA_USER_SAH and pass_input == PASSWORD_SAH:
                    respon_sukses = {
                        "status": "success",
                        "token": TOKEN_RAHASIA
                    }
                    self._send_json(200, respon_sukses)
                else:
                    respon_gagal = {
                        "status": "failed",
                        "message": "Username atau password salah"
                    }
                    self._send_json(401, respon_gagal)

            except json.JSONDecodeError:
                error_json = {"status": "failed", "message": "Format JSON rusak!"}
                self._send_json(400, error_json)
            else:
                rute_salah = {"status": "failed", "message": "Endpoint tidak ditemukan"}
                self._send_json(404, rute_salah)

    def do_GET(self):
        if self.path == "/data":
            header_otorisasi = self.headers.get("Authorization")

            if header_otorisasi and header_otorisasi.startswith("Bearer "):
                try:

```

```

        token_kiriman = header_otorisasi.split(" ")[1]
    except IndexError:
        token_kiriman = ""

    # Memvalidasi token yang dimasukkan secara manual oleh client
    if token_kiriman == TOKEN_RAHASIA:
        respon_data = {
            "status": "success",
            "data": DATA_MAHASISWA
        }
        self._send_json(200, respon_data)
        return

    token_salah = {
        "status": "failed",
        "message": "Token tidak valid"
    }
    self._send_json(401, token_salah)
else:
    rute_salah = {"status": "failed", "message": "Endpoint tidak ditemukan"}
    self._send_json(404, rute_salah)

def run(port=8080):
    alamat_server = ("", port)
    httpd = HTTPServer(alamat_server, RestAPIHandler)
    print("Server REST API berjalan lancar di port 8080...")
    try:
        httpd.serve_forever()
    except KeyboardInterrupt:
        print("\nServer dihentikan.")
        httpd.server_close()

if __name__ == "__main__":
    run()

```

SERVER (client.py) :

```

import urllib.request
import urllib.error
import json

URL_DASAR = "http://localhost:8080"

def alur_simulasi_client():
    print("=== TAHAP 1: REQUEST LOGIN (AUTHENTICATION) ===")
    url_login = URL_DASAR + "/login"

    # -----

```

```

# INPUT MANUAL DATA LOGIN (Ketik Sendiri Mandiri)
# Hint dari dokumen UTS:
# Username: KHOLID GANTENG
# Password: KH0123
# -----

username_manual = input("Masukkan Username: ")
password_manual = input("Masukkan Password: ")

# Memasukkan data ketikan Anda ke dalam paket JSON
data_akun = {
    "username": username_manual,
    "password": password_manual
}

paket_kirim = json.dumps(data_akun).encode("utf-8")

req_login = urllib.request.Request(
    url_login,
    data=paket_kirim,
    headers={"Content-Type": "application/json"},
    method="POST"
)

try:
    with urllib.request.urlopen(req_login) as respon_server:
        body_respon = respon_server.read().decode("utf-8")
        json_respon = json.loads(body_respon)
        print("\nResponse Login Berhasil (200 OK):")
        print(json.dumps(json_respon, indent=4))
        print("\nSilakan salin (copy) token yang muncul di atas!")

except urllib.error.HTTPError as e:
    print("\nLogin Gagal! Username atau Password yang Anda ketik salah.")
    return

print("\n" + "="*50 + "\n")

# =====
# INPUT MANUAL TOKEN (Ketik/Paste Sendiri Mandiri)
# =====
token_manual = input("Masukkan atau tempel (paste) token Anda di sini: ")

print("\n=== TAHAP 2: REQUEST DATA AKADEMIS (AUTHORIZATION) ===")
url_data = URL_DASAR + "/data"

# Mengirimkan token hasil inputan manual tadi ke dalam komponen HTTP Header
req_data = urllib.request.Request(
    url_data,

```

```

        headers={"Authorization": "Bearer " + token_manual},
        method="GET"
    )

    try:
        with urllib.request.urlopen(req_data) as respon_server:
            body_respon = respon_server.read().decode("utf-8")
            json_respon = json.loads(body_respon)
            print("Response Ambil Data Berhasil (200 OK):")
            print(json.dumps(json_respon, indent=4))

    except urllib.error.HTTPError as e:
        print("Gagal Mengambil Data! Token manual yang Anda masukkan salah atau tidak valid.")

if __name__ == "__main__":
    try:
        alur_simulasi_client()
    except urllib.error.URLError:
        print("Error Jaringan: Pastikan file 'server.py' sudah Anda jalankan terlebih dahulu!")

```

5. Implementasikan authentication dan authorization Authentication

Sistem terdistribusi ini memisahkan konsep keamanan ke dalam dua tahapan berlapis secara ketat di sisi server:

A. Authentication (Autentikasi)

- **Letak Kode:** Berada di dalam blok fungsi `do_POST()` pada pemeriksaan kondisi:

Python

```
if username == USERNAME_DB and password == PASSWORD_DB:
```

- **Prinsip Kerja:** Tahapan ini melakukan pemeriksaan identitas digital awal untuk mengenali *siapa* entitas (Client) yang mencoba masuk ke dalam sistem. Proses ini memverifikasi kecocokan kombinasi *username* (KHOLID GANTENG) dan *password* (KHO123) yang dikirim melalui endpoint `/login`. Output akhir dari proses yang berhasil ini adalah penerbitan karcis akses berupa seuntai string unik yaitu `token-kholid-12`.

B. Authorization (Otorisasi)

- **Letak Kode:** Berada di dalam blok fungsi `do_GET()` pada pemeriksaan kondisi:

Python

```
if token == TOKEN_DB:
```

- **Prinsip Kerja:** Tahapan ini memeriksa tingkat hak kepemilikan izin atau hak akses berdasarkan token yang dibawa dan dikirim oleh Client di dalam komponen HTTP Header (`Authorization: Bearer ...`) ketika memanggil endpoint `/data`. Sistem melakukan validasi token ini untuk memastikan bahwa hanya Client dengan token yang sah, benar, dan terdaftar yang diizinkan untuk mengekstrak data sensitif akademis milik **Kholid Mashuri** dari database internal server.